SkillSwap | Project Report

Web Programming (511) - Milestone 1 - Winter 2026 - German International University

1. Team and Project Idea

Team GYMM	ID	Email
Yahia Mohamed	16004934	yahia.ghitas@student.giu-uni.de
Malak Elsadek	16004523	malak.elsadek@student.giu-uni.de
Maya Ramy	16001727	maya.mohamed@student.giu-uni.de
Ghada Haitham	16005525	ghada.maki@student.giu-uni.de

Repository: https://github.com/malakelsadek/skill-swap.git

Project idea. SkillSwap is an online course marketplace ("Learn a skill. Share a skill."). Instructors publish courses and earn a share of every paid sale; students browse, buy courses one by one or through a monthly subscription, and can request refunds. A back-office side (accountant and admin) runs payroll, the department budget, instructor payouts and refund approvals, so the whole life cycle of a course sale is covered in one Angular application.

2. Description of the Application

SkillSwap is a single-page Angular application. The navigation bar and the available pages change with the logged-in role, and route guards protect every restricted page.

- **Visitors:** home page, course catalog (search, category filter, sorting), course details, Log in and Sign up. The sign-up form validates every field (required values, valid email, password of 8+ characters with a digit, picture upload, card details, favourite categories).
- **Students:** cart with order summary, mock checkout (Pay Now / Pay Later), invoices (Pending/Paid), payment history, refund requests with a reason, and a profile page (masked card, preferences, sound setting).
- **Instructors:** dashboard to add, edit and delete their own courses, and an earnings page showing the 70% share of every paid sale (pending and refunded invoices are not counted).
- **Accountant and admin:** department budget with a bar chart, staff payroll (salary, allowances, deductions, net, monthly JSON report), instructor payouts (JSON report), refund approvals (approving marks the invoice as refunded and records it in the budget) and, for the admin, a Users list.
- **Cross-cutting:** AI chatbot that recommends courses, sound-effects toggle, responsive layout with a hamburger menu, lazy-loaded routes, a "Not authorized" page, and 158 Karma/Jasmine specs that all pass.

3. Data Storage: Implementation and Why

There is no backend in this milestone, so all data lives in the browser. Every read and write goes through one **StorageService** that wraps the three mechanisms from the lecture. Each piece of data goes to the mechanism whose lifetime and visibility match how sensitive it is.

Mechanism	What we store	Lifetime / scope	Why, and effect on the user
Local storage	Business data as JSON under sw_ keys: sw_courses, sw_enrollments, sw_invoices, sw_payments, sw_refunds, sw_users, sw_sound; budget entries too.	Until deleted; same browser and origin; survives closing the browser.	Purchases and refunds are still there on the next visit and show up in the instructor and accountant views. Nothing goes over the network, so lists feel instant.
Session storage	sw_session = {"userId":3} (only the id) and the chatbot conversation.	Only while the tab is open; not shared between tabs.	Stays logged in across refreshes but is logged out when the tab closes (safer on shared computers). No password or personal data is kept.
Cookie	sw_remember = username only (e.g. student1), written only when the user ticks "Remember me".	Has an expiry, path "/" and SameSite=Lax.	Recognises a returning user, but only with their consent. A tiny value avoids the ~4 KB limit and extra bytes on every request.

Impact on the user experience

- **Privacy:** the session holds only an id and the cookie only a username, never a password. Only the last 4 card digits are kept (profile shows **** **** **** 4242). The cookie exists only with consent (GDPR, as discussed in the lecture) and the chat history disappears with the tab.
- **Security:** passwords are stored as a hexadecimal hash (modified DJB2), never as plain text; at login the typed password is hashed and compared. A failed login shows one generic message so an attacker cannot tell which part was wrong.
- **Time saving:** users do not re-enter data (purchases persist, the login survives a refresh, the optional cookie remembers returning users). Large data sits in local storage, not cookies, so it is not sent with every request, and lazy-loaded routes make the first page download less code.
- **Reliability:** StorageService returns null instead of crashing on corrupted JSON, seeds demo data only when storage is empty, and keeps session data separate from local data (covered by unit tests).

Limitations. DJB2 is a fast, unsalted, non-cryptographic hash, so it only avoids plain text. Browser storage can be read from DevTools (Figures 1-3), is limited to one browser and a few MB, and the JavaScript-set cookie cannot be HttpOnly. A real version would use a server, salted bcrypt/Argon2 and HttpOnly, Secure, SameSite cookies.

Sample run. A student logs in with "Remember me" (password hashed and compared with sw_users, session = id, cookie = username), adds courses to the cart and checks out (invoice and payment written to local storage), later requests a refund (sw_refunds); when the accountant approves it, the invoice becomes refunded and the budget is updated. All roles read the same records, so every screen below reflects the same data.

4. Screenshots of Sample Runs

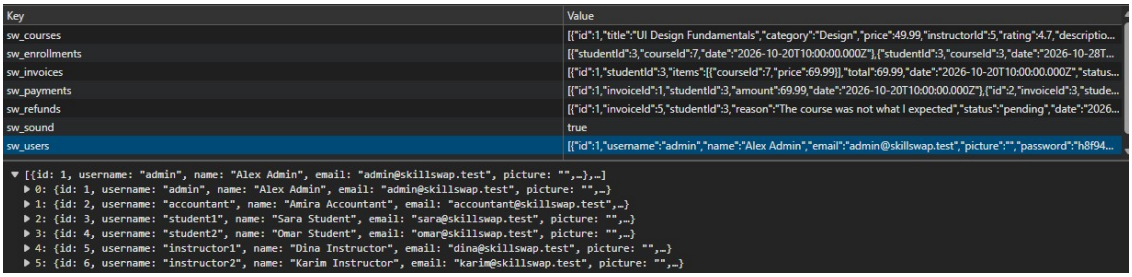


Fig. 1 Local storage: sw_ keys for courses, enrollments, invoices, payments, refunds, users, sound.

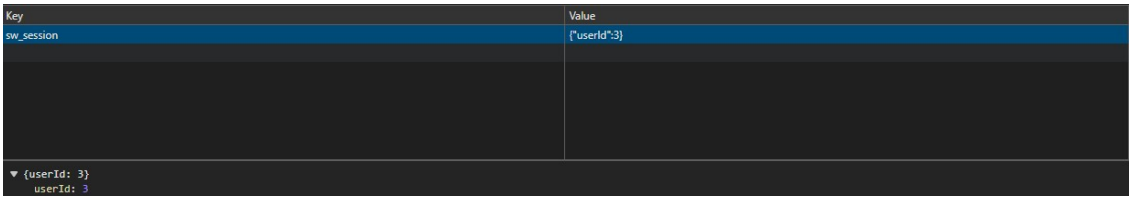


Fig. 2 Session storage: sw_session holds only {"userId":3}.

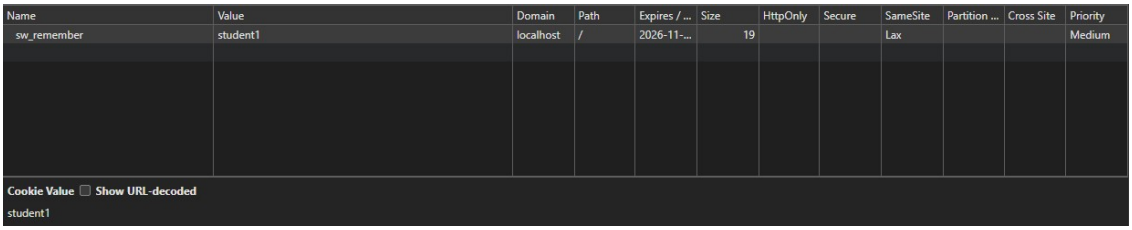
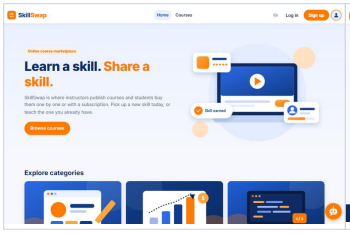
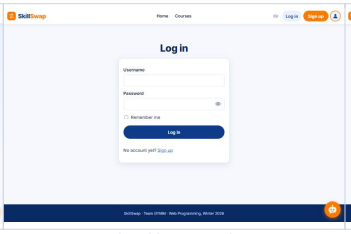


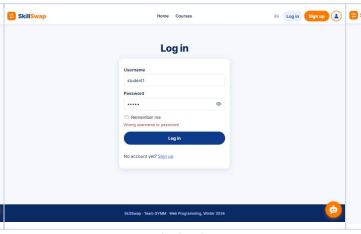
Fig. 3 Cookie: sw_remember = student1 with an expiry, path / and SameSite=Lax.



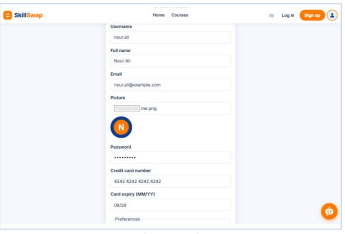
4. Home page



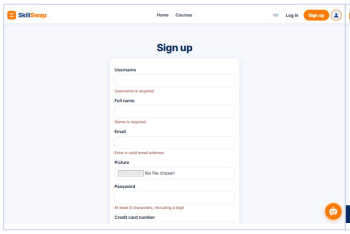
5. Log in with Remember me



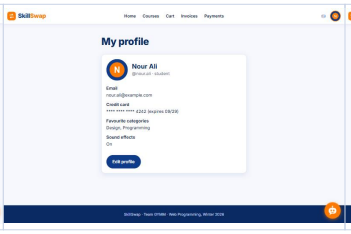
6. Generic login error



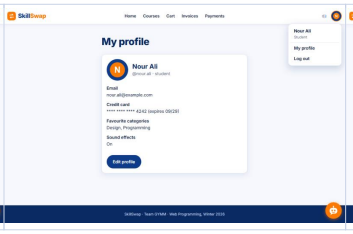
7. Sign up form



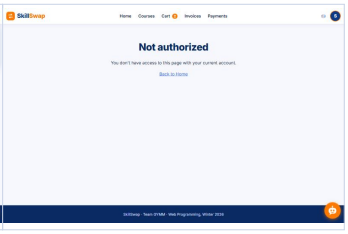
8. Sign up validation



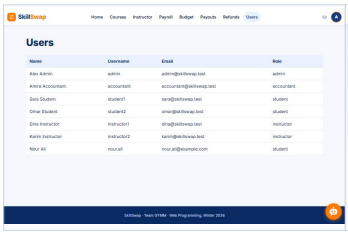
9. Profile, masked card



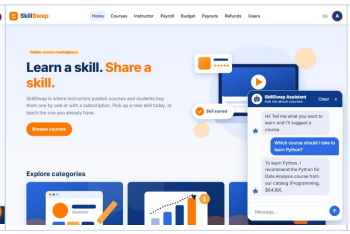
10. Account menu



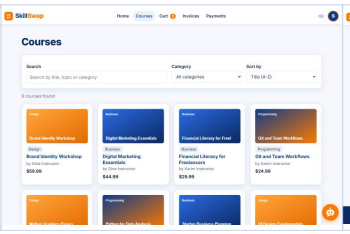
11. Role guard: Not authorized



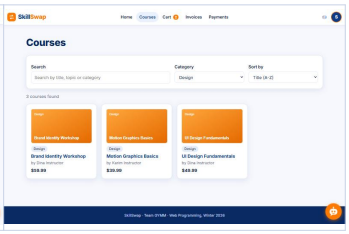
12. Admin: Users list



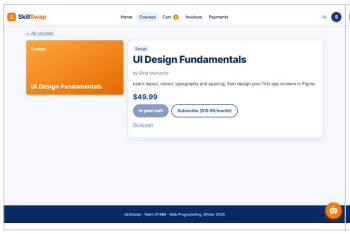
13. Course chatbot



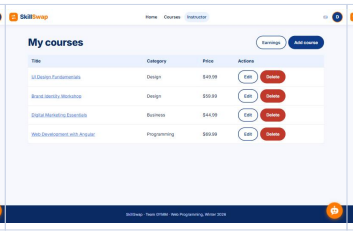
14. Course catalog



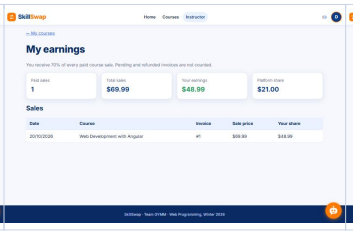
15. Filter: Design



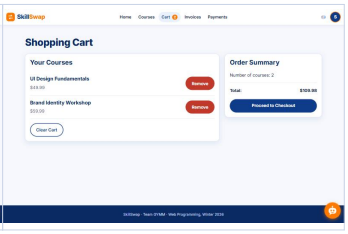
16. Course detail



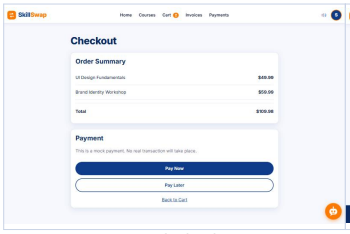
17. Instructor dashboard



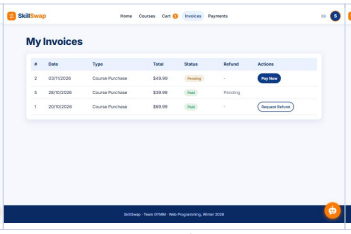
18. Instructor earnings (70%)



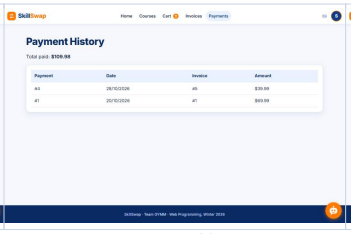
19. Cart and summary



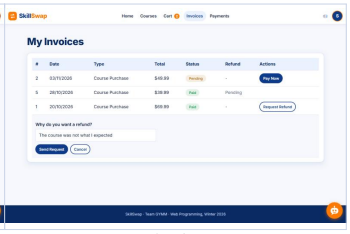
20. Mock checkout



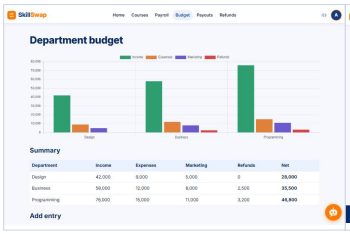
21. Invoices



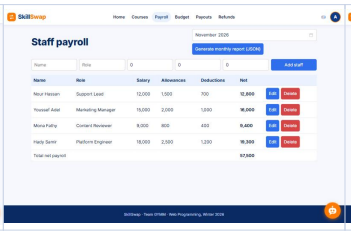
22. Payment history



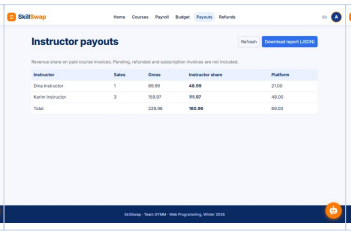
23. Refund request



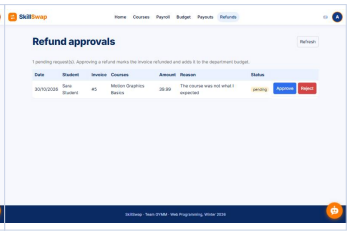
24. Budget chart



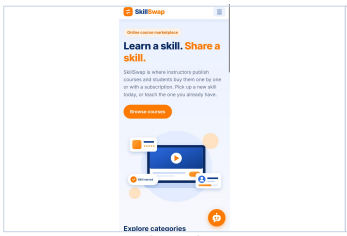
25. Staff payroll



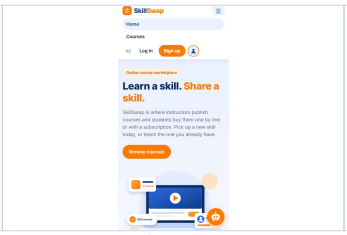
26. Instructor payouts



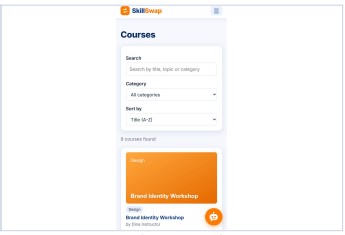
27. Refund approvals



28. Phone: home



29. Phone: menu



30. Phone: catalog



Fig. 31 Lazy loading: the catalog code chunk is requested only when the Courses page is opened.

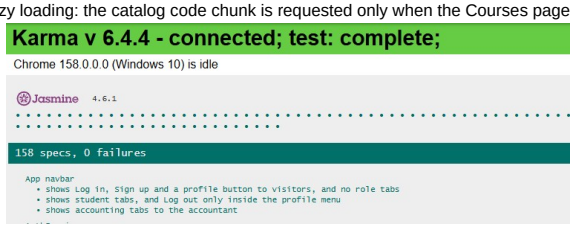


Fig. 32 Karma/Jasmine: 158 specs, 0 failures.